

Lesson 1: Introduction to Program Logic Formulation & Flowcharting

1. Algorithms & Logic Formulation Tools

An **algorithm** is a step-by-step set of instructions designed to perform a specific task or solve a given problem. Two primary tools are used to document program logic:

- **Pseudocode:** A numbered, human-readable list of sequential instructions that outlines program steps without strict syntax rules.
- **Flowchart:** A pictorial/diagrammatic representation that uses standard geometric symbols and flowlines to depict the step-by-step flow and operations of an algorithm.

2. Standard Flowchart Symbols

Symbol Name	Purpose / Function
Terminator	Marks the exact starting (Start) and stopping (End / Stop) points of a process.
Input / Output (I/O)	Represents data entering the system (e.g., Read, Input) or exiting it (e.g., Print, Display).
Process	Represents computational tasks, variable manipulations, or arithmetic calculations (e.g., sum = a + b).
Decision	Evaluates a condition or boolean statement (True / False, Yes / No) to determine branching paths.
Preparation / Initialization	Declares variables, sets constants, or initializes loop counters (e.g., CurrentYear = 2026).
On-Page Connector	Connects separate flowchart segments on the same page to prevent tangled flowlines.
Off-Page Connector	Connects flowchart logic spanning across multiple separate physical pages.

Flowline	Indicates the direction and execution sequence of the program steps.
-----------------	--

Lessons 2 & 3: Introduction to Programming Concepts & Java Fundamentals

1. Core Programming Terminology

- **Computer Program:** A structured collection of instructions executed by a computer to perform specific tasks.
- **Computer Programming:** The systematic process of designing, writing, testing, and maintaining executable instructions.
- **Computer Programmer:** A specialist who writes, tests, and debugs code to build functioning software applications.
- **Programming Language:** A formal language containing instructions understood by both humans and computing devices. Common paradigms include:
 - **Procedural:** Step-by-step procedure execution (e.g., C).
 - **Object-Oriented (OOP):** Organized around reusable classes and objects (e.g., Java).
 - **Scripting:** Interpreted execution often automating environments (e.g., Python, JavaScript).
 - **Logic Programming:** Formulated through formal mathematical logic rules (e.g., Prolog).

2. The 6 Steps in Creating a Computer Program

1. **Defining and Analyzing the Problem:** Identifying real-world needs, root causes, program scope, and required outputs.
2. **Planning the Solution:** Designing logic via pseudocode, flowcharts, or structured models before coding.
3. **Coding the Program:** Translating planned logic into standard, syntactically correct source code.
4. **Testing the Program:** Executing the application to verify behavior:
 - *Alpha Testing:* Internal testing conducted by developers inside the organization.
 - *Beta Testing:* External testing evaluated by selected end users in real-world scenarios.
5. **Debugging:** Locating, isolating, and fixing syntax, logical, and runtime errors.
6. **Final Documentation:** Writing end-user guides, technical manuals, and inline code documentation.

3. Java History & Characteristics

- **Origins:** Conceived in **1991** under Sun Microsystems by the **Green Team** (led by **James Gosling**, alongside Patrick Naughton and Mike Sheridan) under the "Green Project".
- **Naming:** Initially named **Oak**; renamed to **Java** in **1994** because Oak was trademarked. "Java" symbolizes energy, creativity, and productivity.
- **Current Owner:** Sun Microsystems was acquired by **Oracle Corporation**.
- **Key Features:**
 - **WORA (Write Once, Run Anywhere):** Platform independence achieved through virtual machines.
 - **Robust:** Strong memory management and Automatic Garbage Collection (AGC).

```
Java
import java.io.*;           // Package / Library Import

public class SampleCode {   // Access Modifier & Class Name
    public static void main(String[] args) { // Main Method Entry Point
        int num1 = 50;      // Data Type, Identifier, Literal
        int num2 = 50;      // Variable declaration & initialization
        int sum = 0;        // Memory allocation

        sum = num1 + num2;   // Arithmetic Operation Statement
        System.out.println("The sum is: " + sum); // Output with String Concatenation
    }                       // Closing brace for main method
}                          // Closing brace for class
```

2. Element Reference Guide

- **Packages (`import java.io.*`):** Pre-built classes and routines. The wildcard `*` imports all classes contained in the package.
- **Access Modifiers:** Determine class/variable visibility:
 - `public`: Accessible throughout all classes in the project.
 - `private`: Encapsulated and accessible only inside its defining class.
 - `protected`: Accessible within the same package or by subclasses.
 - `default` (no keyword): Package-private visibility.
- **Class & File Naming Agreement:** The name of the `public class` must exactly match the filename saved on disk (e.g., `public class SampleCode` must be saved in `SampleCode.java`).
- **Main Method (`public static void main(String[] args)`):** The designated execution entry point where JVM execution begins. `String[] args` holds command-line string arguments.
- **Data Types:**
 - *Primitive Types*: Built-in scalar values:
 - Integers: `byte` (8-bit), `short` (16-bit), `int` (32-bit), `long` (64-bit).
 - Floating-Point: `float` (single precision), `double` (double precision).
 - Text/Boolean: `char` (single 16-bit Unicode character), `boolean` (`true` or `false`).
 - *Non-Primitive Types*: User-defined references, strings, and collections (e.g., `String`, `Arrays`).
- **Terminator (`;`):** Denotes the completion of an individual statement.
- **Concatenation (`+`):** Joins literal values or variables to text strings within print statements.
- **Output Methods:**
 - `System.out.print()`: Outputs text on the current console line.
 - `System.out.println()`: Outputs text and moves the cursor to a new line.
- **Comments (`//`):** Notes ignored by the Java compiler during compilation.

Lesson 5: Generating User-Defined Input

1. Static Literals vs. User-Defined Input

- **Literal Values:** Hardcoded values specified directly in the code (e.g., `int x = 50;`) which remain static on each execution.
- **User-Defined Input:** Dynamic values supplied at runtime by the user via interactive input streams (e.g., keyboard input).

2. The `Scanner` Class

Found in the `java.util` package, `Scanner` parses primitive types and strings from input streams (`System.in`).

Common `Scanner` Methods:

- `scannerInstance.nextLine()`: Reads an entire string line (including white spaces) until enter is pressed.
- `scannerInstance.next()`: Reads a single whitespace-delimited token/word.
- `scannerInstance.nextInt()`: Scans the next integer value.
- `scannerInstance.nextDouble()`: Scans the next double-precision floating-point value.
- `scannerInstance.nextFloat()`: Scans the next single-precision floating-point value.

3. Implementation Blueprint

Java

```
import java.util.Scanner; // Import the Scanner class

public class GradeAverageCalculator {
    public static void main(String[] args) {
        // Instantiate the Scanner object linked to keyboard input (System.in)
        Scanner input = new Scanner(System.in);

        // String input prompt
        System.out.print("Enter your name: ");
        String studentName = input.nextLine();

        // Numeric input prompts
        System.out.print("Enter your grade in Math: ");
        double math = input.nextDouble();

        System.out.print("Enter your grade in Filipino: ");
        double filipino = input.nextDouble();

        System.out.print("Enter your grade in English: ");
        double english = input.nextDouble();

        System.out.print("Enter your grade in Science: ");
        double science = input.nextDouble();

        System.out.print("Enter your grade in P.E.: ");
        double pe = input.nextDouble();

        // Arithmetic Computation
        double gpa = (math + filipino + english + science + pe) / 5.0;

        // Concatenated output
        System.out.println(studentName + ", your GPA is " + gpa);

        input.close(); // Close the input stream
    }
}
```